

FATEC Ourinhos

Análise e Desenvolvimento de Sistemas — 3.º Semestre

ESTRUTURA DE DADOS

Resumo Completo para Prova

Material de Estudo com Simulado

1. Ponteiros e Alocação Dinâmica em C

Ponteiro é uma variável que armazena um endereço de memória. É o mecanismo central de todas as estruturas dinâmicas em C: sem ponteiros não há listas encadeadas, filas, pilhas ou árvores. Dominar ponteiros significa entender o modelo de memória — stack vs. heap — e o ciclo malloc/free.

Stack vs. Heap

Stack (automático)	Heap (dinâmico)
+-----+	+-----+
int x = 10;	malloc(sizeof(No)) <-- você gerencia
char buf[64];	calloc(n, size)
No *ptr; (8 B) -----> No { dado, *prox }	
+-----+	+-----+
Liberado ao sair	Liberado SOMENTE via free()
da função (automático)	Vazamento se esquecer!

Figura 1 — Stack (local/automático) vs. Heap (dinâmico, controlado pelo programador)

Operações essenciais com ponteiros

```
int valor = 42;
int *ptr = &valor;      // & = endereço de
printf("%d\n", *ptr); // * = desreferenciar (acessar o valor)

// Alocação dinâmica
typedef struct No {
    int dado;
    struct No *proximo;
} No;

No *novo = malloc(sizeof(No)); // aloca no heap
if (novo == NULL) { /* sem memória */ exit(1); }
novo->dado = 10;          // -> eh (*novo).dado
novo->proximo = NULL;
free(novo);              // devolve ao SO — OBRIGATORIO
```

Código 1 — Ponteiro, dereferenciamento e ciclo malloc/free

Dica: Regra de ouro: todo malloc tem um free correspondente. Ponteiro após free deve ser setado a NULL para evitar uso acidental (dangling pointer). Acesso a NULL gera segfault — valide sempre o retorno do malloc.

Complexidade das estruturas — visão geral

Estrutura	Acesso	Busca	Insercao (inicio)	Insercao (fim)	Remocao
Array estatico	O(1)	O(n)	O(n) shift	O(1) amortizado	O(n) shift
Lista encadeada	O(n)	O(n)	O(1)	O(1) c/ ptr fim	O(n) busca + O(1)
Fila (FIFO)	O(n)	O(n)	N/A	O(1) enqueue	O(1) dequeue
Pilha (LIFO)	O(n)	O(n)	O(1) push	N/A	O(1) pop
Lista dupla circular	O(n)	O(n)	O(1)	O(1)	O(1) com no em maos
BST (media)	O(n)	O(log n)	O(log n)	O(log n)	O(log n)

2. Lista Encadeada Simples (AA06)

Lista encadeada simples e uma sequencia de nos onde cada no contem um dado e um ponteiro para o proximo no. O ultimo no aponta para NULL. Diferente do array, os elementos nao ficam em posicoes contiguas de memoria — a conexao e feita pelos ponteiros. Isso torna insercao e remocao O(1) quando se tem o no anterior, mas busca O(n) pois nao ha acesso direto por indice.

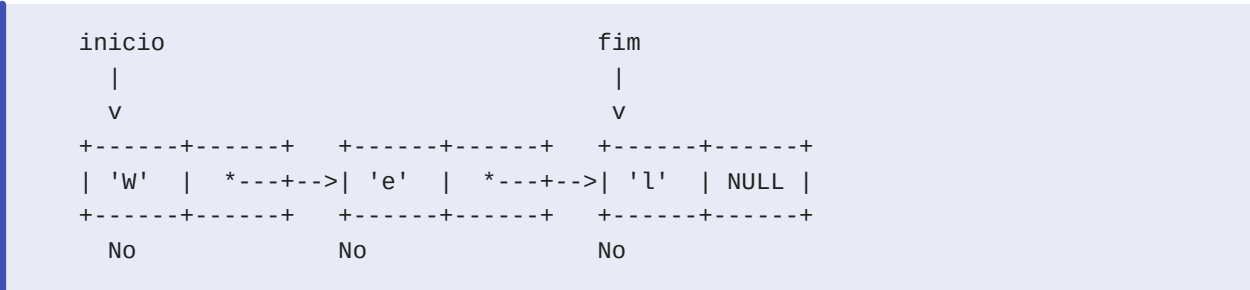


Figura 2 — Lista encadeada simples com ponteiros inicio e fim

Implementacao: insercao no final

```
void inserir(Lista *lista, char c) {
    No *novo = malloc(sizeof(No));
    novo->dado = c;
    novo->proximo = NULL;

    if (lista->inicio == NULL) { // lista vazia
        lista->inicio = novo;
        lista->fim = novo;
    } else {
        lista->fim->proximo = novo; // encadeia no fim
        lista->fim = novo; // atualiza ponteiro fim
    }
}
```

Código 2 — Inserção no final: $O(1)$ porque guardamos ponteiro para o fim

Implementação: remoção de elemento específico

```
void remover(Lista *lista, char c) {
    if (lista->inicio == NULL) return;

    // Caso especial: elemento esta no inicio
    if (lista->inicio->dado == c) {
        No *temp = lista->inicio;
        lista->inicio = lista->inicio->proximo;
        if (lista->inicio == NULL) lista->fim = NULL;
        free(temp);
        return;
    }

    // Percorre buscando o anterior ao alvo
    No *anterior = lista->inicio;
    No *atual = lista->inicio->proximo;
    while (atual != NULL) {
        if (atual->dado == c) {
            anterior->proximo = atual->proximo; // desencadeia
            if (atual == lista->fim) lista->fim = anterior;
            free(atual); // libera
            return;
        }
        anterior = atual;
        atual = atual->proximo;
    }
}
```

Código 3 — Remoção por valor: $O(n)$ busca + $O(1)$ desencadeamento

Dica: Ponto critico da remocao: voce precisa do NO ANTERIOR para ajustar o ponteiro proximo. Por isso percorre com dois ponteiros (anterior + atual). Trate sempre o caso especial do primeiro elemento separadamente.

3. Fila — FIFO (AA07)

Fila e uma estrutura FIFO (First-In, First-Out): o primeiro elemento a entrar e o primeiro a sair. Analogia real: fila de banco, fila de impressao, BFS em grafos. Duas operacoes fundamentais: enqueue (inserir no fim) e dequeue (remover do inicio). Ambas sao $O(1)$ quando se mantem ponteiros para inicio e fim.

ENQUEUE (entrada)DEQUEUE (saida)

||

v v

fim -->[50]->[40]->[30]->[20]->[10]<-- inicio

Estado apos enqueue(60):

fim -->[60]->[50]->[40]->[30]->[20]->[10]<-- inicio

Estado apos dequeue() retorna 10:

fim -->[60]->[50]->[40]->[30]->[20]<-- inicio

Figura 3 — Fila: entrada pelo fim, saida pelo inicio

Implementacao

```
typedef struct No { int dado; struct No *proximo; } No;
typedef struct Fila { No *inicio; No *fim; int tamanho; } Fila;

// Enqueue: O(1)
void enfileirar(Fila *fila, int valor) {
    No *novo = malloc(sizeof(No));
    novo->dado = valor;
    novo->proximo = NULL;
    if (fila->inicio == NULL) { fila->inicio = novo; fila->fim = novo; }
    else { fila->fim->proximo = novo; fila->fim = novo; }
    fila->tamanho++;
}

// Dequeue: O(1)
int desenfileirar(Fila *fila) {
    if (fila->inicio == NULL) return -1; // fila vazia
    No *removido = fila->inicio;
    int valor = removido->dado;
    fila->inicio = fila->inicio->proximo;
    if (fila->inicio == NULL) fila->fim = NULL;
    free(removido);
    fila->tamanho--;
    return valor;
}
```

Código 4 — Fila com lista encadeada: enqueue e dequeue O(1)

Quando usar Fila?

- BFS (Busca em Largura) em grafos: processa vizinhos nível por nível
- Escalonamento de processos Round-Robin: fila de prontos
- Buffer de impressao, requisicoes de rede: ordem de chegada
- Simulacao de filas de atendimento (bancos, call centers)

4. Pilha — LIFO (AA08)

Pilha é uma estrutura LIFO (Last-In, First-Out): o último elemento inserido é o primeiro a ser removido. Analogia: pilha de pratos, pilha de chamadas de funções (call stack). Operações: push (empilhar no topo), pop (desempilhar do topo), peek/top (consultar topo sem remover). Todas O(1).

PUSH 'SP'	PUSH 'RJ'	PUSH 'MG'	POP	PEEK
+-----+	+-----+	+-----+	+-----+	+-----+
topo	topo	topo	topo	topo
[SP]	[RJ]	[MG]<--	[RJ]<--	[RJ]
	[SP]	[RJ]	[SP]	[SP]
		[SP]		
+-----+	+-----+	+-----+	+-----+	+-----+
			retorna MG	retorna RJ
			sem remover	

Figura 4 — Estado da pilha apos push/pop/peek

Implementacao

```
typedef struct No { char dado[3]; struct No *proximo; } No;
typedef struct Pilha { No *topo; int tamanho; } Pilha;

// Push: O(1) – insere como novo topo
void Push(Pilha *pilha, const char *uf) {
    No *novo = malloc(sizeof(No));
    strncpy(novo->dado, uf, 2);
    novo->dado[2] = '\0';
    novo->proximo = pilha->topo; // aponta para o topo atual
    pilha->topo = novo;          // novo vira o topo
    pilha->tamanho++;
}

// Pop: O(1) – remove e retorna o topo
char *Pop(Pilha *pilha) {
    if (pilha->topo == NULL) return NULL; // underflow
    No *removido = pilha->topo;
    static char valor[3];
    strncpy(valor, removido->dado, 3);
    pilha->topo = pilha->topo->proximo;
    free(removido);
    pilha->tamanho--;
    return valor;
}
```

Codigo 5 — Pilha com lista encadeada: push e pop O(1)

Quando usar Pilha?

- Ctrl+Z (desfazer): operacoes empilhadas, desfeitas na ordem inversa
- Avaliacao de expressoes aritmeticas (notacao polonesa reversa — RPN)
- DFS (Busca em Profundidade) em grafos: pilha explicita ou recursao (call stack implicita)
- Verificacao de parenteses balanceados: push '(', pop ao encontrar ')'
- Chamadas de funcao recursivas: o proprio processador usa a call stack

Dica: Diferença chave Pilha vs. Fila: pilha inverte a ordem (LIFO), fila preserva (FIFO). Para implementar 'desfazer', use pilha. Para processar por ordem de chegada, use fila.

5. Lista Duplamente Encadeada Circular (AA09)

Cada nó possui dois ponteiros: próximo (avança) e anterior (retrocede). O último nó aponta de volta para o primeiro (circular), formando um anel. Isso permite: travessia nos dois sentidos, remoção em $O(1)$ quando se tem o nó alvo (sem precisar do anterior), e acesso eficiente ao fim sem percorrer a lista.

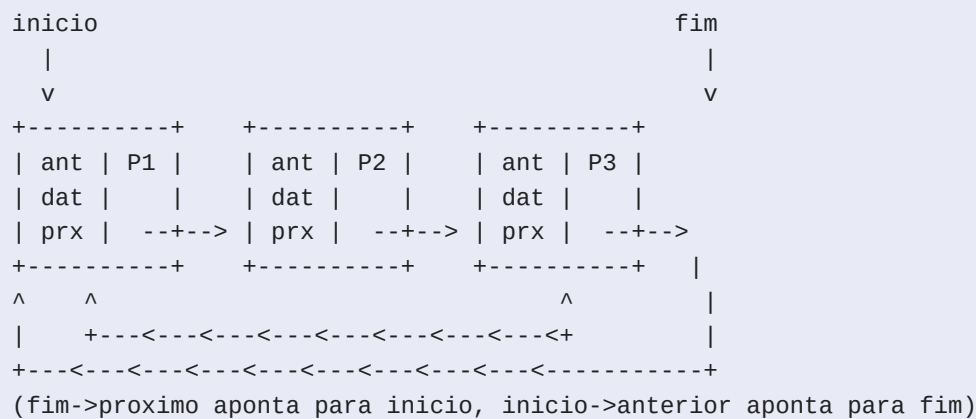


Figura 5 — Lista duplamente encadeada circular

Vantagem: remoção $O(1)$ com o nó em mãos

```
// Com o ponteiro 'alvo' em mãos, remoção é  $O(1)$ :
void removerNo(Lista *lista, No *alvo) {
    if (lista->tamanho == 1) {
        lista->inicio = NULL;
        lista->fim = NULL;
    } else {
        alvo->anterior->proximo = alvo->proximo;
        alvo->proximo->anterior = alvo->anterior;
        if (alvo == lista->inicio) lista->inicio = alvo->proximo;
        if (alvo == lista->fim) lista->fim = alvo->anterior;
    }
    free(alvo);
    lista->tamanho--;
}
```

Código 6 — Remoção $O(1)$: ajusta os 4 ponteiros vizinhos

Inserção na lista circular


```
void inserirElemento(Lista *lista, int cod, float desconto) {
    No *novo = malloc(sizeof(No));
    novo->dado.cod_produto = cod;
    novo->dado.desconto     = desconto;

    if (lista->inicio == NULL) {           // primeiro no
        novo->proximo = novo;              // aponta para si mesmo
        novo->anterior = novo;
        lista->inicio = novo;
        lista->fim    = novo;
    } else {                               // encadeia antes do inicio
        novo->anterior = lista->fim;
        novo->proximo = lista->inicio;
        lista->fim->proximo = novo;
        lista->inicio->anterior = novo;
        lista->fim            = novo;
    }
    lista->tamanho++;
}
```

Codigo 7 — Insercao: ajuste dos 4 links circulares

Travessia circular: cuidado com o loop

```
// Percorre usando do-while para incluir o inicio
void imprimirLista(Lista *lista) {
    if (lista->inicio == NULL) return;
    No *atual = lista->inicio;
    do {
        printf("Cod: %d | %.2f%%\n",
            atual->dado.cod_produto,
            atual->dado.desconto);
        atual = atual->proximo;
    } while (atual != lista->inicio); // para quando volta ao inicio
}
```

Codigo 8 — Travessia circular com do-while

Operacao	Lista Simples	Lista Dupla Circular
Insercao no inicio	O(1)	O(1)
Insercao no fim	O(1) com ptr fim	O(1) com ptr fim
Remocao com no em maos	O(n) — precisa do anterior	O(1) — tem ponteiro anterior
Remocao por valor	O(n)	O(n) busca + O(1) remocao
Travessia invertida	Impossivel	O(n) — percorre via anterior
Memoria por no	dado + 1 ponteiro	dado + 2 ponteiros

6. Recursao e a Pilha de Chamadas

Uma funcao recursiva e aquela que chama a si mesma para resolver um subproblema menor. Toda funcao recursiva precisa de: (1) caso base — condicao de parada que nao faz chamada recursiva; (2) caso recursivo — reduz o problema e chama a funcao com parametro menor. Sem caso base, a recursao nunca para (stack overflow).

```
// Potencia recursiva: base^exp
long long potencia(long long base, int exp) {
    if (exp == 0) return 1;           // caso base
    return base * potencia(base, exp - 1); // caso recursivo
}

// Trace de potencia(2, 4):
// potencia(2,4) = 2 * potencia(2,3)
//               = 2 * 2 * potencia(2,2)
//               = 2 * 2 * 2 * potencia(2,1)
//               = 2 * 2 * 2 * 2 * potencia(2,0)
//               = 2 * 2 * 2 * 2 * 1 = 16
```

Codigo 9 — Potencia recursiva e trace de execucao

A Call Stack durante recursao

Chamada potencia(2, 3):

```
+-----+ <- topo da stack
| potencia(2, 0) | retorna 1
+-----+
| potencia(2, 1) | aguarda, calcula 2*1=2
+-----+
| potencia(2, 2) | aguarda, calcula 2*2=4
+-----+
| potencia(2, 3) | aguarda, calcula 2*4=8
+-----+ <- frame original
| main()         |
+-----+
```

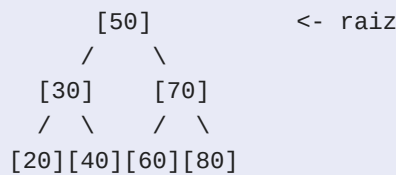
Figura 6 — Pilha de chamadas (call stack) durante recursao

Dica: Recursao de cauda (tail recursion): quando a chamada recursiva e a ULTIMA operacao da funcao (sem multiplicar ou somar o resultado). Compiladores otimizam tail recursion eliminando frames da stack (tail call optimization — TCO). GCC com -O2 faz isso. A potencia acima NAO e tail recursion pois multiplica apos retornar.

7. Arvore Binaria de Busca — BST (Proximo Topico)

BST e uma arvore binaria com a invariante: para todo no, todos os valores na subarvore esquerda sao menores e todos na direita sao maiores. Isso permite busca $O(\log n)$ no caso medio — mas $O(n)$ no pior caso (arvore degenerada, inserida em ordem). Arvores AVL e Rubro-Negras garantem $O(\log n)$ no pior caso com rebalanceamento automatico.

Insercao de 50, 30, 70, 20, 40, 60, 80:



Invariante: esquerda < no < direita
Travessia in-order: 20 30 40 50 60 70 80 (ordem crescente!)

Figura 7 — BST balanceada com 7 nos

Estrutura e operacoes em C

```
typedef struct No {
    int dado;
    struct No *esquerda;
    struct No *direita;
} No;

// Insercao: O(log n) medio
No* arvore_inserir(No *raiz, int dado) {
    if (raiz == NULL) { // posicao encontrada
        No *novo = malloc(sizeof(No));
        novo->dado = dado;
        novo->esquerda = NULL;
        novo->direita = NULL;
        return novo;
    }
    if (dado < raiz->dado)
        raiz->esquerda = arvore_inserir(raiz->esquerda, dado);
    else if (dado > raiz->dado)
        raiz->direita = arvore_inserir(raiz->direita, dado);
    // igual: ignorar (sem duplicatas)
    return raiz;
}

// Travessia in-order (esq -> raiz -> dir) = ordem crescente
void arvore_in_order(No *raiz) {
    if (raiz == NULL) return;
    arvore_in_order(raiz->esquerda);
    printf("%d ", raiz->dado);
    arvore_in_order(raiz->direita);
}

// Destruir: post-order (libera filhos antes da raiz)
void arvore_destruir(No *raiz) {
    if (raiz == NULL) return;
    arvore_destruir(raiz->esquerda);
    arvore_destruir(raiz->direita);
    free(raiz);
}
```

Codigo 10 — BST: insercao recursiva e travessias

As tres travessias

Travessia	Ordem de visita	Resultado na BST do diagrama	Uso tipico
In-order	Esq -> Raiz -> Dir	20 30 40 50 60 70 80	Listar em ordem crescente
Pre-order	Raiz -> Esq -> Dir	50 30 20 40 70 60 80	Serializar/clonar arvore
Post-order	Esq -> Dir -> Raiz	20 40 30 60 80 70 50	Destruir arvore (free)

8. Guia de Escolha de Estrutura

A escolha da estrutura de dados certa e uma das decisoes de design mais importantes. Use a tabela abaixo como guia rapido para questoes de prova e situacoes reais.

Cenario / Problema	Estrutura Ideal	Justificativa
Implementar Ctrl+Z (desfazer)	Pilha (LIFO)	Operacoes empilhadas, desfeitas na ordem inv
Fila de impressao / atendimento	Fila (FIFO)	Ordem de chegada preservada, enqueue/dequeue
Navegar historico do browser (voltar/avancar)	Lista dupla	Navegacao em ambos os sentidos O(1)
Playlist circular de musicas	Lista dupla circular	Volta ao inicio automaticamente
Busca rapida por valor	BST / Hash	O(log n) BST, O(1) amortizado Hash
Remover elemento frequentemente (posicao conhecida)	Lista dupla	Remocao O(1) com ponteiro no no
Acesso por indice (arr[i])	Array	O(1) acesso direto — listas sao O(n)
BFS em grafo	Fila	Processa nos nivel por nivel
DFS em grafo / expressoes	Pilha	Profundidade primeiro, LIFO

Dica: Questao classica de prova: 'qual estrutura para implementar Ctrl+Z?' Resposta: PILHA. Cada acao e empilhada (push). Desfazer = pop. Refazer = segunda pilha. Nunca use array para isso — insercao/remocao no meio e O(n) shift.

SIMULADO

Questão 1 (Conceitual)

Explique a diferença entre alocação de memória na stack e no heap em C. Por que estruturas de dados dinâmicas (listas, árvores) usam heap e não stack? O que acontece se você esquecer de chamar `free()` em um nó alocado com `malloc()`?

Questão 2 (Código)

O código abaixo contém um bug grave envolvendo ponteiros. Identifique o problema e reescreva a função corretamente. No* `criar_no(int valor)` { No no; // declarado na stack! no.dado = valor; no.proximo = NULL; return &no; // retorna endereço de variável local } // Chamada: No *novo = criar_no(42); // novo->dado = ??? // comportamento indefinido

// Escreva sua resposta aqui:

Questão 3 (Conceitual)

Qual a complexidade de tempo das operações abaixo em cada estrutura? Justifique cada resposta. a) Busca de um elemento em lista encadeada simples com N nós. b) Inserção no início de uma lista encadeada simples. c) Remoção do primeiro elemento de uma fila com ponteiro de início. d) Pop do topo de uma pilha. e) Remoção de um nó específico em lista duplamente encadeada, dado o ponteiro para o nó.

Questão 4 (Raciocínio)

Trace a execução da função `potencia(3, 3)` mostrando o estado da call stack em cada chamada recursiva. Qual é o valor retornado? Identifique o caso base e o caso recursivo na função abaixo: `long long potencia(long long base, int exp) { if (exp == 0) return 1; return base * potencia(base, exp - 1); }`

Questão 5 (Código)

Implemente a funcao lista_buscar para uma lista encadeada simples de inteiros. A funcao deve retornar o ponteiro para o no que contem o valor buscado, ou NULL se nao encontrar. Use a struct abaixo: typedef struct No { int dado; struct No *proximo; } No; typedef struct { No *inicio; } Lista; No* lista_buscar(Lista *lista, int valor);

```
// Escreva sua resposta aqui:
```

Questão 6 (Conceitual)

Qual estrutura de dados voce usaria para cada cenario abaixo? Justifique. a) Sistema de desfazer/refazer (Ctrl+Z / Ctrl+Y) de um editor de texto. b) Fila de processos prontos no escalonador Round-Robin de um SO. c) Armazenar o historico de paginas de um browser com navegacao voltar/avancar. d) Verificar se uma expressao '({[]})' tem parenteses balanceados.

Questão 7 (Múltipla Escolha)

Sobre lista encadeada simples versus array de tamanho fixo, qual afirmativa e CORRETA? a) Array tem insercao no inicio $O(1)$ pois basta atualizar um indice. b) Lista encadeada tem acesso ao elemento do meio em $O(1)$ via aritmetica de ponteiros. c) Array ocupa menos memoria por elemento pois nao armazena ponteiros. d) Lista encadeada e mais eficiente que array para acesso sequencial em todos os casos. e) Remover o ultimo elemento de uma lista encadeada simples e $O(1)$ com ponteiro para o fim.

Questão 8 (Raciocínio)

Dado o codigo abaixo que opera sobre uma pilha, qual o valor impresso pelo printf final? Trace a execucao passo a passo mostrando o estado da pilha apos cada operacao. Pilha *p = criarPilha(); Push(p, "MS"); Push(p, "DF"); Push(p, "RJ"); Push(p, "PR"); char *t = Top(p); Push(p, Pop(p)); Push(p, "SP"); Push(p, Top(p)); Pop(p); Pop(p); printf("%s\n", Top(p));

GABARITO COMENTADO

Questão 1 (Conceitual):

Stack: memória alocada automaticamente para variáveis locais, liberada ao sair do escopo da função. Tamanho fixo e limitado (~1-8 MB tipicamente). Heap: alocação manual via malloc/calloc, persiste além do escopo da função, tamanho limitado pela memória disponível. Estruturas dinâmicas precisam de heap porque: (1) tamanho não é conhecido em tempo de compilação; (2) elas precisam sobreviver além da função que os criou; (3) a lista pode crescer/encolher em runtime. Sem free(): memory leak — o SO nunca recupera a memória até o processo terminar. Em programas longos, isso exaure a RAM.

Questão 2 (Código):

Bug: a função retorna o endereço de uma variável local (no stack). Ao retornar, o frame da função é destruído e aquela memória pode ser sobrescrita a qualquer momento. Acessar novo->dado é comportamento indefinido (undefined behavior) — o programa pode parecer funcionar ou crashar aleatoriamente. Correção: `No* criar_no(int valor) { No *no = malloc(sizeof(No)); // aloca no heap if (no == NULL) return NULL; no->dado = valor; no->proximo = NULL; return no; // retorna ponteiro para memória persistente }`

Questão 3 (Conceitual):

a) $O(n)$: no pior caso, percorre todos os N nós até encontrar ou chegar ao NULL. Não há acesso aleatório (diferente de array). b) $O(1)$: basta criar o novo nó, apontar seu próximo para o início atual, e atualizar o ponteiro de início. Nenhum percurso necessário. c) $O(1)$: o ponteiro início já aponta para o primeiro nó. Basta salvar o valor, avançar o início para o próximo, e liberar o nó removido. d) $O(1)$: o ponteiro topo já aponta para o elemento a remover. e) $O(1)$: com o ponteiro no nó, ajusta-se apenas os 4 ponteiros vizinhos (anterior->proximo e proximo->anterior). Sem necessidade de percurso.

Questão 4 (Raciocínio):

Caso base: `exp == 0`, retorna 1 (sem chamada recursiva). Caso recursivo: retorna `base * potencia(base, exp-1)`. Trace de `potencia(3, 3)`: `potencia(3,3) -> 3 * potencia(3,2)` `potencia(3,2) -> 3 * potencia(3,1)` `potencia(3,1) -> 3 * potencia(3,0)` `potencia(3,0) -> retorna 1 (caso base)`. Desfazendo: `potencia(3,1)=3*1=3`; `potencia(3,2)=3*3=9`; `potencia(3,3)=3*9=27`. Resultado final: 27. A call stack cresce 4 frames (`potencia 3,2,1,0`) e depois desempilha retornando o resultado acumulado.

Questão 5 (Código):

No* lista_buscar(Lista *lista, int valor) { No *atual = lista->inicio; while (atual != NULL) { if (atual->dado == valor) return atual; // encontrou atual = atual->proximo; } return NULL; // nao encontrado } Complexidade: $O(n)$ — percorre no maximo N nos. Nao ha como fazer melhor em lista encadeada pois nao ha acesso por indice (diferente de array, onde busca binaria seria $O(\log n)$).

Questão 6 (Conceitual):

a) Duas pilhas (LIFO): uma para acoes feitas (Ctrl+Z faz pop), outra para desfeitas (Ctrl+Y faz pop da segunda e push na primeira). LIFO e natural pois desfazemos na ordem inversa de execucao. b) Fila circular (FIFO): cada processo entra no fim e sai do inicio. Apos usar seu quantum, volta ao fim. FIFO garante fairness (ordem de chegada). c) Lista duplamente encadeada: botao Voltar = anterior, botao Avancar = proximo. Insercao de nova pagina = inserir apos posicao atual e descartar o resto. d) Pilha: para cada abre-parentese push; para cada fecha-parentese, pop e verifica se corresponde. Ao final, pilha deve estar vazia. $O(n)$ tempo, $O(n)$ espaco.

Questão 7 (Múltipla Escolha):

Alternativa CORRETA: c. Array armazena apenas os dados, sem overhead de ponteiros. Lista encadeada armazena dado + 1 ponteiro (simples) ou dado + 2 ponteiros (dupla) por no. a) ERRADA: insercao no inicio de array e $O(n)$ pois requer shift de todos os elementos. b) ERRADA: lista encadeada e $O(n)$ para acessar elemento do meio — sem aritmetica de ponteiros. d) ERRADA: array tem melhor localidade de cache para acesso sequencial (elementos contiguos na RAM). e) ERRADA: remover o ultimo numa lista simples e $O(n)$ — precisa percorrer ate o penultimo para atualizar o ponteiro fim (sem ponteiro anterior disponivel).

Questão 8 (Raciocínio):

Trace passo a passo: Push MS -> [MS]. Push DF -> [DF, MS]. Push RJ -> [RJ, DF, MS]. Push PR -> [PR, RJ, DF, MS]. Top(p) = PR (nao remove). Pop(p) = PR, depois Push(PR) -> [PR, RJ, DF, MS] (estado inalterado). Push SP -> [SP, PR, RJ, DF, MS]. Top(p) = SP, Push(SP) -> [SP, SP, PR, RJ, DF, MS]. Pop(p) remove SP -> [SP, PR, RJ, DF, MS]. Pop(p) remove SP -> [PR, RJ, DF, MS]. Top(p) = PR. printf imprime: PR